I chose the default project: jack in a box simulation.

For this project, I developed the following system that references the frames used:

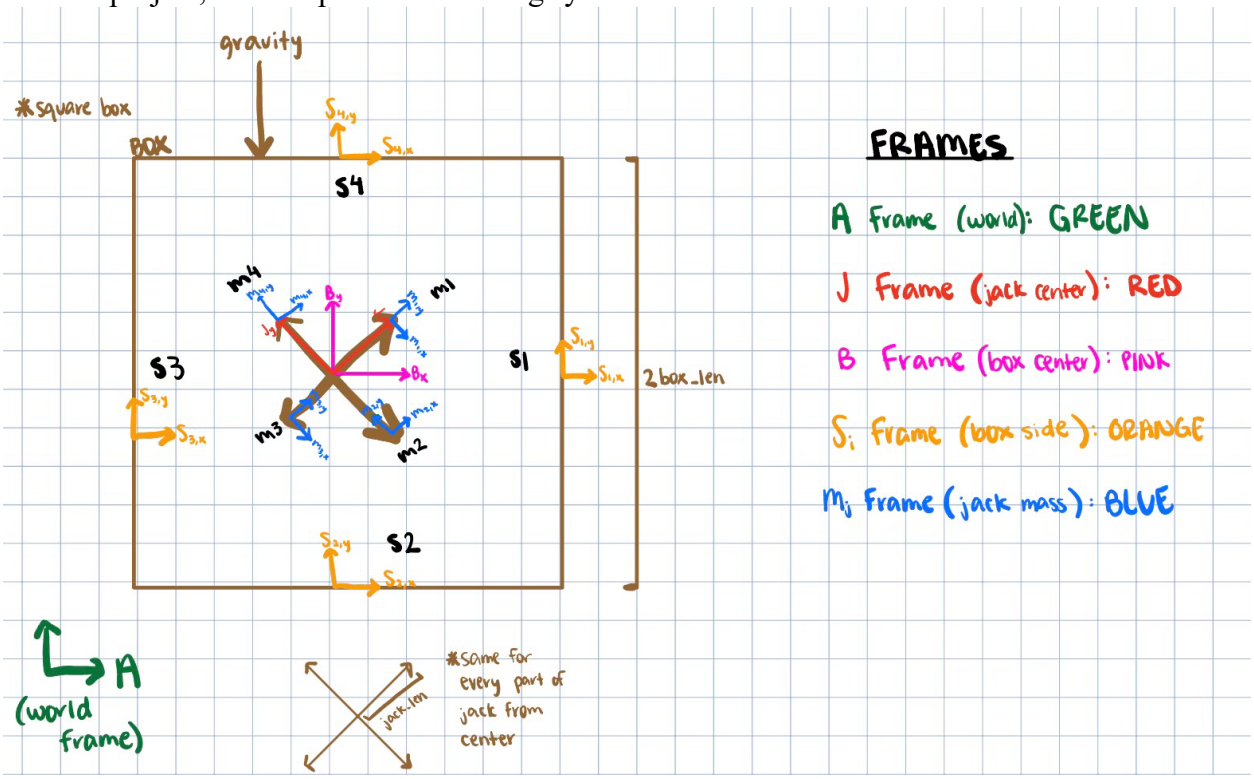


Figure 1: Jack in a box system drawing with frames labelled.

In accordance to this system, homogeneous transformations were made to relate one frame to another. This can be seen in the table below:

Frame Transformation	Description	How to Identify in Code
A to B	Box frame (B) expressed in world frame (A)	g_ab
A to J	Jack frame (J) expressed in world frame (A)	g_aj
B to S_i	Side frame (S_i, sides of box) expressed in box frame (B)	g_b_s{i} *#i for numbers 1-4*
J to M_j	Mass frame (M_j, ends/masses of jack) expressed in jack frame (J)	g_j_m{j} *#j for numbers 1-4*
A to S_i	Side frame (S_i) expressed in world frame (A)	g_ab_s[i] = g_ab @ g_b_s[i]
A to M_j	Mass frame (M_j) expressed in world frame (A)	g_aj_s[j] = g_aj @ g_j_m[j]
S_i to M_j	Mass frame (M_j) expressed in side frame (S_i)	g_rel[i][j] = mat_inv(g_ab @ g_b_s[i]) @ (g_aj @ g_j_m[j])

Table 1: Rigid body transformations between frames.

For the completion of this project, calculations were necessitated, starting with the Euler-Lagrange equations. Using the transformations listed above, g_ab and g_aj, the orientation/positioning of the box and frames are obtained. When a transformation is differentiated and multiplied by its inverse, this gives the velocity ($\hat{V}$). With the box, for example, this is done by taking the inverse of g_ab and multiplying this matrix by the derivative of g_ab. Once unhat is used, the 4 by 4 matrix converts to a 6 component velocity vector. This process is then repeated for g_aj (the jack.) Now that the velocities are obtained, the kinetic energy ($.5V^TIV$) and potential energy (mgy) of both the box and jack are calculated. The Lagrange is then completed by subtracting the total potential energy from the total kinetic energy. At this point, every calculation needed for the Euler-Lagrange (EL) equation is obtained. As the EL equation is defined as the following:

$$\frac{d}{dt}\left(\frac{\partial \mathrm{L}}{\partial \dot{q}}\right) - \frac{\partial \mathrm{L}}{\partial \mathrm{q}} = 0$$

Each component of the equation is calculated in the code using a jacobian and time derivative. With there being outside external force influence, however, this equation equates to F_ext instead.

Diving further into the external forces calculated, it was necessary to introduce a gravity counter (grav_counter) and shaking force (box_torque) onto the box. The force counteracting gravity evidently displays as a load in the positive y-direction to keep the box in place. The torque, on the other hand, is about the z-axis, sinusoidal, and meant to simulate a hand shaking the box. When these components are put into a force vector (F_ext), they represent the right-hand side of the EL system.

Moving onto the calculation of constraints (the phis), phi depicts how close in contact a side of the box is with one of the masses (ends) of the jack. In short, if phi equals 0, the jack and side of the box are in contact and impact is occurring. As each mass has been correlated to each side through frame reference (g_rel)[i][j], the constraints can build on this. The right and left sides (1 and 3) are translated using (0,3), and the top and bottom sides (2 and 4) are translated using (1,3). These values are then stacked into one vector to get the total constraint vector (tot_phi_dum.) Once lambdify is used, the symbolic vector is now numerical.

As was said above, impact update laws rely on the constraints as phi at 0 determines if impact occurs. As g_rel already creates coordinates normal to the sides of the box (x for right and left, and y for top and bottom), the gradient of phi of q determines the same normal direction in vector form. The linear momentum equation is determined in this direction using the following equation, with the plus referring to after impact:

$$\left(\frac{\partial \mathrm{L}}{\partial \dot{q}}\right)^{+} - \left(\frac{\partial \mathrm{L}}{\partial \dot{q}}\right) = \lambda \nabla \Phi(q)$$

And each equation component is calculated in the code. The Hamiltonian before and after impact are subtracted and equal to 0 to express a perfectly elastic collision. The update_imp function then solves for post-impact velocities and lambda (the impulse magnitude) to instantaneously flip velocities after impact. The find_impact function detects if an impact has occurred by evaluating every constraint value for each mass and side pair (with 0, within a threshold of .01, being an impact found.)

Overall, the animation starts with the jack in the center of the box and the box beginning to rotate (due to s_init). As the jack falls from gravity, it starts to hit different sides and bounces off. The jack rotates as it goes between impacts due to the shaking torque on the box. From the simulation and what has been explained above, the behavior is reasonable: gravity acts on the jack, the box shakes from torque but roughly stays in place due to grav_counter, and the constraints are respected as impacts are detected and the jack moves accordingly post-impact.